Diffusion Guidance via Density Ratio Estimation

Sam Cleary-McHarg

February 2026



Contents

1 Introduction **3**
- 1.1 Problem Motivation . . . 3
- 1.2 Report Structure . . . 3

2 Background **3**
- 2.1 Density Ratio Estimation . . . 4
  - 2.1.1 Density Ratio Formulation . . . 4
  - 2.1.2 Probabilistic Ratio Estimation . . . 4
  - 2.1.3 Least Squares Ratio Estimation . . . 6
- 2.2 Diffusion models and Guidance . . . 7
  - 2.2.1 Diffusion model foundations . . . 7
  - 2.2.2 Diffusion Guidance Theory . . . 8
- 2.3 Machine Learning Foundations . . . 9
  - 2.3.1 Neural Network Architectures . . . 9
  - 2.3.2 Kernels Methods . . . 10
  - 2.3.3 Representation Learning . . . 10
- 2.4 Evaluation Metrics . . . 11
  - 2.4.1 Feature Extraction . . . 11
  - 2.4.2 Precision, Recall, and Coverage . . . 11
  - 2.4.3 Distribution Distance (MMD and FID) . . . 12

3 Methodology **12**
- 3.1 Probabilistic Implementation . . . 12
  - 3.1.1 Density Ratio Estimation . . . 12
  - 3.1.2 Network Architecture . . . 13
- 3.2 Least Squares Ratio Estimation . . . 13
  - 3.2.1 Basis Function Construction . . . 13
  - 3.2.2 Dimensionality Reduction . . . 13
  - 3.2.3 Analytical Solution . . . 14
- 3.3 Diffusion and Guided Sampling . . . 15
  - 3.3.1 Diffusion Model Setup . . . 15
  - 3.3.2 Guidance Integration . . . 15
- 3.4 Evaluation Framework . . . 17
  - 3.4.1 Feature Extraction . . . 17
  - 3.4.2 Metrics . . . 17

4 Experiments **17**
- 4.1 Dataset Analysis . . . 17
  - 4.1.1 Dataset Descriptions . . . 17
  - 4.1.2 Dataset Samples . . . 18
- 4.2 Double Moons . . . 18
- 4.3 MNIST numbers . . . 20
- 4.4 CIFAR-10 . . . 22
- 4.5 Ablation Study . . . 25

5 Discussion **26**
- 5.1 Performance Comparison . . . 26
- 5.2 Future Work . . . 27

6 Conclusion **27**

A Appendix **29**
- A.1 uLSIF Least Squares Analytical Solution . . . 29

1 Introduction

1.1 Problem Motivation

Diffusion models (Ho et al., 2020; Song et al., 2021) are able to create state-of-the-art quality images, outperforming prior approaches like GANs(Dhariwal and Nichol, 2021).

However, once a diffusion model has been trained, the foundational model can only sample from the distribution it was trained on. Diffusion model neural networks also require a high parameter count, so retraining can take a long time or even be computationally unfeasible. A conditional distribution could be obtained from a class label, such as 8, 'animal', or an image quality characteristic like brightness. The solution researched in this report is to estimate a ratio between two different distribution density functions, and use this density ratio to guide the sampling process towards the conditional distribution.

Due to the constantly evolving state of machine learning, dataset labels can be updated to better reflect the data, or some datasets can get entirely new sets of labels. For example, a dataset that has classes for animals might get a new whole set of labels for animal colour or background scenery. For this case, a much more efficient way of estimating the density ratio would be helpful. This report explores a probabilistic method and a least squares method (which can be solved analytically (Kanamori et al., 2009)) to estimate the density ratio.

1.2 Report Structure

The remainder of this report will include 5 main sections: background, methodology, experiments, discussion and conclusion. The background will mathematically present the density ratio estimation techniques, diffusion model guidance and the evaluation metrics. This will ensure the reader has the knowledge and definitions to understand how these methods are used in later sections. The methodology will give technical details on how these concepts are implemented and the setup used in the investigation. The experiments section will present the effectiveness of the guidance methods using images, tables and figures to give many different angles of how well the methods performed. The discussion will report on the experiments as a whole and talk about future plans and research. The conclusion will give an overall reflection on the entire report.

2 Background

This background will mathematically define the density ratio and the estimation methods. Then, we will further define the diffusion model and how the density ratio can be used to guide the model towards a conditional distribution. Machine learning foundations like neural network architecture, kernel methods, and representation learning techniques will be included, as well as the evaluation metrics used in the experiments.

2.1 Density Ratio Estimation

The density ratio is a ratio between the probability density functions of two different distributions(Sugiyama et al., 2012). The density functions of high-dimensional data distributions, like image datasets, are intractable. Intractable functions are not computationally viable due to the complexity of the underlying distribution. So, methods for estimating the density ratio are used instead; these methods can be probabilistic or direct. The probabilistic method will involve using a logistic regressor to output the probability of a sample being from either distribution. The direct estimation method will involve using a linear combination of basis functions to directly estimate the ratio.

2.1.1 Density Ratio Formulation

Let $X \in \mathbb{R}^d$ be the domain of all possible d-dimensional real values and $\boldsymbol{x} \in X$ be a vector in the reals. Any probability density function p with an asterisk p^* represents the theoretical form of the density function. Let $\{\boldsymbol{x}_i^{\text{nu}}\}_{i=1}^{n^{\text{nu}}}$ be n^{nu} i.i.d samples from the numerator distribution with density function $p^*_{\text{nu}}(\boldsymbol{x})$, and let $\{\boldsymbol{x}_i^{\text{de}}\}_{i=1}^{n^{\text{de}}}$ be n^{de} samples from the denominator distribution with density function $p^*_{\text{de}}(\boldsymbol{x})$. The denominator density function $p^*_{\text{de}}(\boldsymbol{x})$ must be strictly positive. The theoretical density ratio $r^*(\boldsymbol{x})$ is a simple ratio between the two densities, defined as:

$$r^*(\boldsymbol{x}) = \frac{p^*_{\text{nu}}(\boldsymbol{x})}{p^*_{\text{de}}(\boldsymbol{x})}, \quad \forall \boldsymbol{x} \in X \tag{1}$$

2.1.2 Probabilistic Ratio Estimation

This section derives the formulas needed to understand how a logistic regressor can be used to approximate a density ratio, and defines the loss function used to minimise the logistic regressor.

Bayesian Representation Letting y take two labels 'nu' and 'de' to represent $\boldsymbol{x}$ sampled from the numerator and denominator distribution, respectively. Let the two densities be written conditionally as

$$p^*_{\text{nu}}(\boldsymbol{x}) = q^*(\boldsymbol{x}|y = \text{'nu'}), \quad p^*_{\text{de}}(x) = q^*(\boldsymbol{x}|y = \text{'de'})$$

These densities model the full $\boldsymbol{x}$ for each class, which is not intractable for high-dimensional data. To solve this problem, Bayes' Theorem is used to condition on the sample $\boldsymbol{x}$ instead. This approach allows for the conditional densities to be approximated when a large number of samples are used.

$$q^*(\boldsymbol{x}|y = \text{'nu'}) = \frac{q^*(y = \text{'nu'}|\boldsymbol{x})q^*(\boldsymbol{x})}{q^*(y = \text{'nu'})}$$
$$q^*(\boldsymbol{x}|y = \text{'de'}) = \frac{q^*(y = \text{'de'}|\boldsymbol{x})q^*(\boldsymbol{x})}{q^*(y = \text{'de'})}$$

Substituting these conditional densities into the original density ratio equation 1. The intractable $q^*(\boldsymbol{x})$ terms will cancel leaving the new form which can be approximated for high dimensional data (Kanamori et al., 2010; Sugiyama et al., 2012). In the specific case that the probability of getting a sample from either distribution is the same $q^*(y = \text{'nu'}) = q^*(y = \text{'de'})$, the ratio can be further simplified to

$$r^*(\boldsymbol{x}) = \frac{q^*(y = \text{'nu'}|\boldsymbol{x})q^*(y = \text{'de'})}{q^*(y = \text{'nu'})q^*(y = \text{'de'}|\boldsymbol{x})} = \frac{q^*(y = \text{'nu'}|\boldsymbol{x})}{q^*(y = \text{'de'}|\boldsymbol{x})} \tag{2}$$

Probabilistic Estimation The conditional densities of the numerator and denominator distribution must sum to $1 = q^*(y = \text{'nu'}|\boldsymbol{x}) + q^*(y = \text{'de'}|\boldsymbol{x})$, using equation 2 the conditional densities $q^*(y = \text{'nu'}|\boldsymbol{x})$ and $q^*(y = \text{'de'}|\boldsymbol{x})$ can be written in the form of the density ratios

$$q^*(y = \text{'nu'}|\boldsymbol{x}) = \frac{r^*(\boldsymbol{x})}{1 + r^*(\boldsymbol{x})} \tag{3}$$

$$q^*(y = \text{'de'}|\boldsymbol{x}) = \frac{1}{1 + r^*(\boldsymbol{x})} \tag{4}$$

Let a neural network f be parametrised by $\boldsymbol{\theta}^{r_1}$, inputting $\boldsymbol{x} \in X$, the neural network outputs logit $z = f(\boldsymbol{x}; \boldsymbol{\theta}^{r_1})$. Since the density ratio must be non-negative, an exponential function is used to model the ratio

$$r(\boldsymbol{x}; \boldsymbol{\theta}^{r_1}) = \exp(f(\boldsymbol{x}; \boldsymbol{\theta}^{r_1}))$$

Substituting the modelled density ratio $r(\boldsymbol{x}; \boldsymbol{\theta}^{r_1})$ into the theoretical equation 3, derives the approximation of the density function $q^*(y = \text{'nu'}|\boldsymbol{x})$ in the form of the sigmoid function $\sigma(z)$

$$q(y = \text{'nu'}|\boldsymbol{x}; \boldsymbol{\theta}^{r_1}) = \frac{r(\boldsymbol{x}; \boldsymbol{\theta}^{r_1})}{1 + r(\boldsymbol{x}; \boldsymbol{\theta}^{r_1})} = \frac{\exp(f(\boldsymbol{x}; \boldsymbol{\theta}^{r_1}))}{1 + \exp(f(\boldsymbol{x}; \boldsymbol{\theta}^{r_1}))} = \sigma(f(\boldsymbol{x}; \boldsymbol{\theta}^{r_1})) \tag{5}$$

Let $\hat{\boldsymbol{\theta}}^{r_1}$ be the trained parameters for a logistic regressor trained to predict the probability of a sample being from the numerator distribution. Then the log-density ratio is simply the logit outputted by the neural network, $z = f(\boldsymbol{x}, \hat{\boldsymbol{\theta}}^{r_1})$.

Loss Objective Letting $q(y = \text{'de'}|\boldsymbol{x}; \boldsymbol{\theta}^{r_1})$ be the estimated conditional density for the denominator distribution. Then the optimal parameters $\hat{\boldsymbol{\theta}}^{r_1}$ can be found by maximizing (Kanamori et al., 2010; Sugiyama et al., 2012)

$$\hat{\boldsymbol{\theta}}^{r_1} = \text{argmax}_{\boldsymbol{\theta}^{r_1}} \left[\sum_{i=1}^{n^{\text{nu}}} \log q(y = \text{'nu'}|\boldsymbol{x}_i^{nu}; \boldsymbol{\theta}^{r_1}) + \sum_{i=1}^{n^{\text{de}}} \log q(y = \text{'de'}|\boldsymbol{x}_i^{de}; \boldsymbol{\theta}^{r_1}) \right]$$

The summations are maximised when the predicted probabilities $q(y = \text{'nu'}|\boldsymbol{x}^{\text{nu}}; \boldsymbol{\theta}^{r_1})$ approach 1. Through gradient descent, the optimal parameters $\hat{\boldsymbol{\theta}}^{r_1}$ can be found that maximise this function.

2.1.3 Least Squares Ratio Estimation

The proceding section will explore the key parts of the derivation of the least squares importance fitting (LSIF). After, it will derive the unconstrained version that will be used throughout the investigation due to the analytical properties of the solution.

Linear Model The density ratio $r^*(\boldsymbol{x})$ can be modelled by a linear combination of b basis functions $\{\varphi_l(\boldsymbol{x})\}_{l=1}^{b}$ such that

$$\varphi_l(\boldsymbol{x}) \geq 0 \quad \text{for all } \boldsymbol{x} \in X \text{ and } l \in \{1, 2, \cdots, b\}$$

The vector of parameters $\boldsymbol{\theta}^{r_2} = (\theta_1^{r_2}, \theta_2^{r_2}, \cdots, \theta_b^{r_2})^\top$ and vector of basis functions $\boldsymbol{\varphi(x)} = (\varphi_1(\boldsymbol{x}), \varphi_2(\boldsymbol{x}), \cdots, \varphi_b(\boldsymbol{x}))$ (Kanamori et al., 2009) will be key to formulating the summation in terms of matrix multiplication in later parts.

$$r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2}) = \sum_{l=1}^{b} \theta_l^{r_2} \varphi_l(\boldsymbol{x}) \tag{6}$$

Least Squares Objective The loss function is the squared error between the theoretical ratio and the linear combination, weighted by the denominator density function $p_{\text{de}}^*(\boldsymbol{x})$(Kanamori et al., 2009). This integral is similar to the continuous version of the expectation formula, and is chosen due to the differentiable properties of quadratic functions.

$$J_0(\boldsymbol{\theta}^{r_2}) = \frac{1}{2} \int (r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2}) - r^*(\boldsymbol{x}))^2 p_{\text{de}}^*(\boldsymbol{x}) d\boldsymbol{x} \tag{7}$$

Next, fully expanding equation 7, ignoring the terms that do not depend on $\boldsymbol{\theta}^{r_2}$, then substituting equation 6 and the theoretical density ratio equation 1 into $J_0(\boldsymbol{\theta}^{r_2})$ results in the improved loss function $J(\boldsymbol{\theta}^{r_2})$(full steps in appendix A)

$$J(\boldsymbol{\theta}^{r_2}) = \frac{1}{2} \sum_{l,l'=1}^{b} \theta_l^{r_2} \theta_{l'}^{r_2} \int \varphi_l(\boldsymbol{x}) \varphi_{l'}(\boldsymbol{x}) p_{\text{de}}(\boldsymbol{x}) \, d\boldsymbol{x} - \sum_{l=1}^{b} \theta_l^{r_2} \int \varphi_l(\boldsymbol{x}) p_{\text{nu}}(\boldsymbol{x}) \, d\boldsymbol{x} \tag{8}$$

As the integrals in equation 8 are the continuous form of the expectation, they can be approximated by calculating an average of the samples $\{\boldsymbol{x}_i^{nu}\}_{i=1}^{n^{\text{nu}}}$ and $\{\boldsymbol{x}_i^{de}\}_{i=1}^{n^{\text{de}}}$. The result is then an element of a vector $\hat{\boldsymbol{h}}$ and matrix $\hat{\boldsymbol{H}}$ indexed by the basis functions $\varphi_l(\boldsymbol{x})$ used

$$\hat{\boldsymbol{h}}_l = \frac{1}{n^{\text{nu}}} \sum_{i=1}^{n^{\text{nu}}} \varphi_l(\boldsymbol{x}_i^{\text{nu}}),$$

$$\hat{H}_{l,l'} = \frac{1}{n^{\text{de}}} \sum_{j=1}^{n^{\text{de}}} \varphi_l(\boldsymbol{x}_j^{\text{de}}) \varphi_{l'}(\boldsymbol{x}_j^{\text{de}}).$$

Lastly, the loss function 8 can be reduced to its matrix form using the sample average vector $\hat{\boldsymbol{h}}$ and matrix $\hat{\boldsymbol{H}}$, (see appendix A).

$$\hat{J}(\boldsymbol{\theta}^{r_2}) = \frac{1}{2}(\boldsymbol{\theta}^{r_2})^T \hat{\boldsymbol{H}} \boldsymbol{\theta}^{r_2} - \hat{\boldsymbol{h}}^T \boldsymbol{\theta}^{r_2} \tag{9}$$

Analytical Solution The general least squares importance fitting (LISF) loss objective includes a regularisation term $\lambda \mathbf{1}_b^\top \boldsymbol{\theta}^{r_2}$, where $\lambda \in \mathbb{R}$ is optimised to stop overfitting by forcing the optimal solution to keep the parameter sizes small. The density ratio must be positive, and as the basis functions are already defined to be positive $\varphi_l(\boldsymbol{x}) > 0$, the parameters $\boldsymbol{\theta}^{r_2}$ must also be greater than 0

$$\min_{\boldsymbol{\theta}^{r_2} \in \mathbb{R}^b} \left[\frac{1}{2}(\boldsymbol{\theta}^{r_2})^\top \hat{\boldsymbol{H}} \boldsymbol{\theta}^{r_2} - \hat{\boldsymbol{h}}^\top \boldsymbol{\theta}^{r_2} + \lambda \mathbf{1}_b^\top \boldsymbol{\theta}^{r_2} \right], \quad \boldsymbol{\theta}^{r_2} > 0 \tag{10}$$

This loss function can be minimised through iterative methods, but the constraint $\boldsymbol{\theta}^{r_2} > 0$, means that an analytical solution is not possible, so the author ignores it (Kanamori et al., 2009). As a result, the regularisation term can be negative, which would create a non-convex solution. To rectify this, a quadratic version is used $\frac{\lambda}{2}(\boldsymbol{\theta}^{r_2})^\top \boldsymbol{\theta}^{r_2}$, forming the unconstrained least squares importance fitting objective (uLSIF)

$$\min_{\boldsymbol{\theta}^{r_2} \in \mathbb{R}^b} \left[\frac{1}{2}(\boldsymbol{\theta}^{r_2})^\top \hat{\boldsymbol{H}} \boldsymbol{\theta}^{r_2} - \hat{\boldsymbol{h}}^\top \boldsymbol{\theta}^{r_2} + \frac{\lambda}{2}(\boldsymbol{\theta}^{r_2})^\top \boldsymbol{\theta}^{r_2} \right] \tag{11}$$

Calculating the derivative analytically and solving equation 11 using linear algebra techniques for the optimal solution (full steps in appendix A), yields the closed-form solution for the parameters.

$$\hat{\boldsymbol{\theta}}^{r_2} = (\hat{\boldsymbol{H}} + \lambda \boldsymbol{I}_b)^{-1} \hat{\boldsymbol{h}} \tag{12}$$

2.2 Diffusion models and Guidance

This section will redefine the useful formulas and notation regarding the diffusion model explored in the group part of the project, so they can be related to throughout this report. The guidance theory section will involve deriving the main guidance formula that will be used throughout the experiments.

2.2.1 Diffusion model foundations

Notation Let $t \in \mathbb{Z}_{\geq 0}$ be a non-negative integer timestep of the sample $\boldsymbol{x}_t$. At timestep $t = 0$, $\boldsymbol{x}_0$ is a clean sample from the data distribution, and let $T \in \mathbb{Z}_{>0}$ be the maximum timestep $0 \leq t \leq T$. Let β_t be the variance defined by a variance schedule, let $\alpha_t = 1 - \beta_t$, and $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$.

Denoising Formula Let $\boldsymbol{\epsilon}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon})$ be the neural networks' predicted noise, then the computed mean takes the form(Dhariwal and Nichol, 2021)

$$\boldsymbol{\mu}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon}) = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon}) \right) \tag{13}$$

Let$\boldsymbol{I}$ be an identity matrix and let $\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ be vector of i.i.d Gaussian noise, using equation 13 the slightly less noised image $\boldsymbol{x}_{t-1}$ can be predicted using

$$\boldsymbol{x}_{t-1} = \boldsymbol{\mu}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon}) + \sqrt{\beta_t} \boldsymbol{z} \tag{14}$$

Diffusion Model Score The score $\hat{\boldsymbol{s}}_t$ of a diffusion model(Dhariwal and Nichol, 2021) can be approximated using the neural networks predicted error $\boldsymbol{\epsilon}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon})$ from a sample $\boldsymbol{x}_t$ by

$$\hat{\boldsymbol{s}}_t = \frac{\boldsymbol{\epsilon}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon})}{\sqrt{1 - \bar{\alpha}_t}} \tag{15}$$

2.2.2 Diffusion Guidance Theory

This section shows how the score of the diffusion model can be altered using a score computed from the density ratio to steer the sampling process towards the conditional distribution.

Modified Score Function Let $\boldsymbol{\theta}^r$ be general parameters, not specific to direct or probabilistic estimation, and let $\hat{\boldsymbol{\theta}}^r$ be general trained parameters for the density ratio estimation. The density ratio estimation $r(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^r)$ is a predicted ratio between two different probability densities, the numerator and denominator. A logarithm is applied to the density ratio $\log r(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^r)$ to give the density ratio logarithmic properties like monotonicity. The gradient of a vector points in the direction of steepest increase, so the gradient of the log-ratio $\nabla_{\boldsymbol{x}_t} \log r(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^r)$ will result in a vector $\boldsymbol{s}_t^{\text{nu}}$ that maximises the numerator probability density function.

$$\boldsymbol{s}_t^{\text{nu}} = \nabla_{\boldsymbol{x}_t} \log r(\boldsymbol{x}_t, \hat{\boldsymbol{\theta}}^r) \tag{16}$$

Using the predicted score $\hat{\boldsymbol{s}}_t$ from the error prediction 15, and the conditional score $\boldsymbol{s}_t^{\text{nu}}$ calculated from the density ratio 16, a guided score function $\tilde{\boldsymbol{s}}_t$ can be calculated. Let $w \in \mathbb{R}_{\geq 0}$ be a scalar determining the strength of the guidance towards the conditional distribution.

$$\tilde{\boldsymbol{s}}_t = \hat{\boldsymbol{s}}_t + w \cdot \boldsymbol{s}_t^{\text{nu}} = \frac{\boldsymbol{\epsilon}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon})}{\sqrt{1 - \bar{\alpha}_t}} + w \cdot \nabla_{\boldsymbol{x}_t} \log r(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^r) \tag{17}$$

Using equation 15, let the guided error be $\tilde{\boldsymbol{\epsilon}}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon}) = -\sqrt{1 - \alpha_t} \tilde{\boldsymbol{s}}_t$, and define the new guided mean as

$$\tilde{\boldsymbol{\mu}}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon}) = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \tilde{\boldsymbol{\epsilon}}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon}) \right)$$

Practical Form Using the guided mean $\tilde{\boldsymbol{\mu}}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^\epsilon)$ and substituting in for the guided score $\tilde{\boldsymbol{\epsilon}}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^\epsilon) = \sqrt{1-\bar{\alpha}}\tilde{\boldsymbol{s}}$, then expanding the expression using equation 17

$$\begin{aligned}
\tilde{\boldsymbol{\mu}}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^\epsilon) &= \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\tilde{\boldsymbol{\epsilon}}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^\epsilon)\right) \\
&= \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\left[\sqrt{1-\bar{\alpha}_t}\left(\frac{\boldsymbol{\epsilon}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^\epsilon)}{\sqrt{1-\bar{\alpha}_t}} + w \cdot \nabla_{x_t} \log r(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^r)\right)\right]\right) \\
&= \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^\epsilon)\right) + w \cdot \frac{1-\alpha_t}{\sqrt{\alpha_t}} \cdot \nabla_{x_t} \log r(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^r))
\end{aligned}$$

In practice, $\sqrt{\alpha_t}$ is very close to 1 because $\alpha_t = 1 - \beta_t$ and the variance schedule β_t is usually set to very small values. Also, a guidance weight w is usually optimised for the best score metric, so the approximation $\sqrt{\alpha_t} \approx 1$ can be made, and the guided mean can be defined by

$$\tilde{\boldsymbol{\mu}}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^\epsilon) = \boldsymbol{\mu}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^\epsilon) + w\beta_t \nabla_{\boldsymbol{x}_t} \log r(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^r)$$

Finally, the ddpm sampling process step towards the conditional distribution can be defined by

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^\epsilon)\right) + w\beta_t \nabla_{x_t} \log r(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^r) + \sqrt{\beta_t}\boldsymbol{z}$$

2.3 Machine Learning Foundations

This section will cover the machine learning foundations used in the methodology. The neural network architectures used for the logistic regressor and diffusion model will be covered. The kernels and representation learning techniques will be mathematically explained.

2.3.1 Neural Network Architectures

The three architectures get more complicated and build on the one explained prior to it.

Multi-layer Perceptron The multi-layer perceptron (MLP) is one of the simplest deep-learning networks, consisting of basic linear layers and ReLU activation functions. Each linear layer uses a matrix of weights $\boldsymbol{W} \in \mathbb{R}^{m \times d}$ and a bias weight $\boldsymbol{b} \in \mathbb{R}^m$, forming the equation $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$. The ReLU activation $\text{ReLU}(\boldsymbol{x}) = \max(0, \boldsymbol{x})$ adds non-linearity, enabling the neural network to model non-linear functions. The MLP has multiple layers to create multiple levels of abstraction.

Convolutional Neural Network The convolutional neural network (CNN) is used when the data has spatial information, for example, image datasets. CNNs consist of layers of convolutional layers, ReLU activations and max-pools. The convolutional layer uses a filter of weights of some size F, $\boldsymbol{W} \in \mathbb{R}^{F \times F}$ that convolves over the input; this enables the neural network to gain spatial information about the image. The max-pool halves the height and width of the input by outputting the max value in each 2 by 2 block, allowing the convolutional layers to function at different levels of spatial abstraction.

U-net The U-net architecture consists of three different parts: the down-sampling channels, up-sampling channels and the bottleneck. The down-sampling channels usually consist of convolutional layers, ReLU activation and a maxpool for halving the height and width of the input image. At each down-sample, the number of feature maps is doubled to account for the smaller height and width. The up-sample channels use transposed convolutions, which double the height and width. Another feature of U-nets is the residual connections, which either concatenate or add the down-sampling outputs to the respective up-sampling channel, so that useful information that may have been lost in the down-sampling channels may still be used.

2.3.2 Kernels Methods

Kernel methods are symmetric functions that can express data in high dimensions.

Gaussian Kernel The Gaussian kernel is defined by the similarity between two points $\boldsymbol{x}$ and $\boldsymbol{x}'$ by using the Euclidean distance.

$$k(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\frac{\|\boldsymbol{x} - \boldsymbol{x}'\|^2}{2\sigma^2})$$

By taking the Gaussian kernel of a point $\boldsymbol{x}$, you can express one vector as the similarity to k usually randomly chosen points $\{\boldsymbol{x}'_i\}_{i=1}^k$. Therefore, expressing the original vector as a k-dimensional vector of distances to other points. The Gaussian kernel

2.3.3 Representation Learning

Representation learning involves useful techniques that compress an input into a lower dimension by ignoring the noise and extracting the features that best represent the input vector.

Autoencoder The autoencoder (Hinton and Salakhutdinov, 2006) is one of the most powerful methods of reducing the dimension of high-dimensional spaces. An autoencoder has two main sections: an encoder and a decoder. The encoder takes an input image, and through convolutional layers and an output layer,

outputs a k-dimensional latent vector. The decoder then takes that latent vector as input and reconstructs the image. Through minimising the mean-squared error between the clean and reconstructed images, the encoder is forced to compress the most useful features from the input image into the latent vector so the decoder can recreate the image as best as possible.

Principal Component Analysis Principal Component Analysis (PCA) (Jolliffe, 2002) is a method of dimensionality reduction, which is very efficient. The method starts by normalising the data to stop larger ranges dominating variables with small ranges. Next, the covariance matrix is calculated over each variable. The covariance matrix represents the correlation between each variable and every other variable. This covariance matrix has insight into which variables are correlated to others, and hence provides little information about the data. The eigenvectors and eigenvalues of the covariance matrix are then calculated; the eigenvectors, called principal components, are a new basis that maximises the variance. Each eigenvector's respective eigenvalue gives the strength of the eigenvalue. By sorting the eigenvalues and choosing the best k respective principal components, the dimension of the data can be reduced.

2.4 Evaluation Metrics

The mathematical explanation behind the evaluation metrics is important to understand what the metrics are actually measuring.

2.4.1 Feature Extraction

Image generation quality cannot be easily assessed by comparing the pixels of the real and sampled images. For example, slight differences in the background colour of MNIST numbers generation could score low even if the actual number is very accurate. A feature extractor can be used to compare the features of the images, like for MNIST numbers: stroke width, number size, and number form. A feature extractor usually consists of the feature part and the output layer. The feature part involves convolutional layers, a flatten and a linear layer to compress into an n-dimensional feature vector. The output layer is used to compress the n-dimensional feature vector into a vector with however many classes the data has, so the cross-entropy loss can be used to train the model. Once trained, the feature part can be used to output a feature vector to fairly compare the real and sampled images.

2.4.2 Precision, Recall, and Coverage

Coverage For point-maps, which are datasets formed of thousands of independent low-dimensional points, coverage is the simplest way of assessing the quality. Coverage can be used to assess the proportion of a distribution of points that lie within some radius of any points in another distribution of points. The

xy-coverage is the proportion of sampled points in the real distribution, a measurement of accuracy. The yx-coverage is the proportion of real samples in the distribution of generated samples.

Precision and Recall The precision and recall are metrics for separating the quality and diversity of the generated images, useful for comparing the trade-off. This metric uses the k-nearest neighbours radius to be able to create a proportion of samples close to another sample. By using the k-nearest neighbours (k-nn), the metric is not biased to the area of the distribution with high density. The precision is then calculated as the proportion of generated samples that are within the k-nn radius of each real sample. The recall is the proportion of real samples that are within a k-nn radius of the generated samples.

2.4.3 Distribution Distance (MMD and FID)

Maximum Mean Discrepancy Maximum Mean Discrepancy MMD^2 is a distance metric between two distributions. The distance between the two distributions uses a Gaussian Kernel for the similarity. The metric calculates expected similarity within the generated samples , similarity between the real samples, and the similarity between the real and generated samples.

$$\text{MMD}^2 = \mathbb{E}[k(X, X')] + \mathbb{E}[k(Y, Y')] - 2\mathbb{E}[k(X, Y)] \tag{18}$$

A smaller MMD implies better sample generations due to the expected similarity score between the X and Y increasing to 1 as X and Y become closer.

Fréchet Inception Distance FID uses the mean of the real images $\bar{\boldsymbol{x}}$ and the mean of the fake images $\bar{\boldsymbol{x}}_0$ where $t = 0$, and the covariance of real and fake images, $\boldsymbol{\Sigma}$ and $\boldsymbol{\Sigma}_0$. The trace of a matrix (summing of the diagonal elements) is defined by the function tr.

$$\text{FID} = \|\bar{\boldsymbol{x}} - \bar{\boldsymbol{x}}_0\|^2 + \text{tr}\left(\boldsymbol{\Sigma} + \boldsymbol{\Sigma}_0 - 2(\Sigma\Sigma_0)^{\frac{1}{2}}\right) \tag{19}$$

3 Methodology

The methodology is set up according to the three different datasets used in the investigation. Specifically, Toy: Double Moons, Structured: MNIST, Complex: CIFAR. For more information about the datasets, see (Section 4.1.1)

3.1 Probabilistic Implementation

3.1.1 Density Ratio Estimation

To be able to train a neural network with the functionality of outputting logits, I used a sigmoid activation function $\sigma(\boldsymbol{z}) = \frac{1}{1-\exp(-\boldsymbol{z})}$ to convert the logits into class probabilities.

I split the dataset into the two categories, the conditional and non-conditional distribution. Then, by training the neural network to calculate the probability of the numerator distribution by minimising the cross entropy loss, I was able to then use the logits outputted by the neural network as an estimated density ratio, as shown in the background section.

3.1.2 Network Architecture

Multi-layer Perceptron The multi-layer perceptron had 2 linear layers, each with 16 neurons, each with a ReLU activation function for non-linearity. The neural network was then trained for 100 epochs with a batch size of 2048 due to the MLP only being used on very low-dimensional data.

Convolutional Neural Networks The convolutional neural network had 4 convolution layers, each with a max-pool and a ReLU activation function. The time embedding was concatenated before the output layer to allow for a time-aware prediction of logits, increasing the accuracy.

3.2 Least Squares Ratio Estimation

This section explains the process I used for analytically solving for the parameters $\boldsymbol{\theta}^{r_2}$, including the dimensionality reduction and basis functions, due to them being directly used before the analytical solution part.

3.2.1 Basis Function Construction

Gaussian Kernel I used the Gaussian Kernel to make my data points more expressive. For data with 2 dimensions, there was not enough information to successfully create a guidance signal. So, I used a Gaussian kernel to express each 2-dimensional data point as a 100-dimensional point using the similarity to 100 randomly chosen data points from the numerator distribution. Randomly choosing 100 data points from the numerator distribution is in line with the paper's recommendations(Kanamori et al., 2009)

Algorithm 1 Gaussian Kernel, given a matrix of the numerator dataset $\boldsymbol{X}^{\text{nu}} \in \mathbb{R}^{n^{\text{nu}} \times d}$ as the conditioned dataset, and number of components k

Input: Numerator data matrix $\boldsymbol{X}^{\text{nu}} \in \mathbb{R}^{n^{\text{nu}} \times d}$, number of components k
Sample 100 random vectors $\boldsymbol{X}^{\text{nu}}_{\text{rand}} \in \mathbb{R}^{100 \times d} \leftarrow [\boldsymbol{x}^{\text{nu}}_1, \boldsymbol{x}^{\text{nu}}_2, \cdots, \boldsymbol{x}^{\text{nu}}_{100}] \sim \boldsymbol{X}^{\text{nu}}$
Calculate distances $\boldsymbol{X}^{\text{cdist}} \in \mathbb{R}^{n \times 100} \leftarrow \text{cdist}(\boldsymbol{X}^{n \times d}, \boldsymbol{X}^{\text{nu}}_{\text{rand}})$
Calculate Gaussian Similarity $\boldsymbol{X}_{\text{new}} \in \mathbb{R}^{n \times 100} \leftarrow \exp(-\frac{1}{2\sigma^2}\boldsymbol{X}^{\text{cdist}})$

3.2.2 Dimensionality Reduction

Dimensionality reduction is very important for forming the analytical solution because each output $\varphi_l(\boldsymbol{x})$ will be considered in the analytical solution, so solv-

ing for $\hat{\boldsymbol{\theta}}^r$ on the most useful features instead of noise is necessary. The different dimensionality reduction techniques I used were Principal Component Analysis (PCA) and an autoencoder (AE).

Algorithm 2 Principal Component Analysis (PCA), given a matrix of the denominator dataset $\boldsymbol{X}^{\text{de}} \in \mathbb{R}^{n^{\text{de}} \times d}$ as the unconditioned dataset, and number of components k

Input: Denominator data matrix $\boldsymbol{X}^{\text{de}} \in \mathbb{R}^{n \times d}$, number of components k
Compute mean $\bar{\boldsymbol{x}}^{\text{de}} \leftarrow \frac{1}{n^{\text{de}}} \sum_{i=1}^{n^{\text{de}}} \boldsymbol{x}^{\text{de}_i}$
Compute standard deviation $\sigma^{\text{de}} \leftarrow (\frac{1}{n^{\text{de}}-1} \sum_{i=1}^{n^{\text{de}}} (\boldsymbol{x}_i^{\text{de}} - \bar{\boldsymbol{x}}^{\text{de}})^2)^{\frac{1}{2}}$
Normalise the data $\boldsymbol{X}_{\text{norm}} \leftarrow \frac{\boldsymbol{X} - \bar{\boldsymbol{x}}^{\text{de}}}{\boldsymbol{\sigma}^{\text{de}}}$
Calculate the covariance $\boldsymbol{X}^{\text{cov}} \leftarrow \frac{1}{n^{\text{de}}-1} \boldsymbol{X}_{\text{norm}}^{\top} \boldsymbol{X}_{\text{norm}}$
Get eigenvectors and values $\boldsymbol{v}_i$ and λ_i
Sort eigenvalues and reorder eigenvectors accordingly
Concatenate eigenvectors for components $\boldsymbol{C} \leftarrow [\boldsymbol{v}_1, \cdots, \boldsymbol{v}_k]$
Dimension reduction $\boldsymbol{X}_{\text{new}} \in \mathbb{R}^{n \times k} \leftarrow X_{\text{norm}} \boldsymbol{C}$

Principal Component Analysis

Autoencoder My autoencoder had a latent dimension of 128, which was enough to represent the features of the image datasets. The encoder consisted of 3 convolutional layers with a stride of 2, so the image height and width would halve at each layer, with feature maps $\{32, 64, 128\}$. Also, a flatten and a linear layer, which compressed the vector to a 128-dimensional latent vector was used. The decoder then reversed the encoder setup using transposed convolutions to the dimensions of the input into the encoder. During training, the autoencoder inputted randomly noised images using the same forward process as the diffusion model, but the loss function used clean images, so the autoencoder wouldn't try to fit the parameters to noise. The autoencoder had a time-embedding using a linear layer to map the time to a vector with the same number of dimensions as the number of feature maps in the convolutional layers. The autoencoder was trained for 50 epochs, which was enough for the MSE loss function to stop having noticeable improvements.

3.2.3 Analytical Solution

Analytical Solution By using the unconstrained least squares importance fitting, I was able analytically solve for $\hat{\boldsymbol{\theta}}^{r_2}$. Calculating $\hat{\boldsymbol{H}}$ and $\hat{\boldsymbol{h}}$ was efficient, due to them being a simple average. Letting $\boldsymbol{X}^{\text{nu}} \in \mathbb{R}^{n^{\text{nu}} \times d}$ be the numerator distribution and $\boldsymbol{X}^{\text{nu}} \in \mathbb{R}^{n^{\text{de}} \times d}$ be the denominator distribution, then the new datasets that were transformed to have a new dimension k. Letting $\boldsymbol{\varphi}(\boldsymbol{X}_{\text{new}}^{\text{nu}})$ and $\boldsymbol{\varphi}(\boldsymbol{X}_{\text{new}}^{\text{de}})$ be vectors for the results of the functions on the numerator and

denominator distribution respectively.

$$\hat{\boldsymbol{H}} = \frac{1}{n^{\text{de}}} \boldsymbol{\varphi}(\boldsymbol{X}^{\text{de}})^\top \boldsymbol{\varphi}(\boldsymbol{X}^{\text{de}}) \quad \hat{\boldsymbol{h}} = \frac{1}{n^{\text{nu}}} \sum \boldsymbol{\varphi}(\boldsymbol{X}^{\text{nu}})$$

Using the analytical solution derived in the background, and a regularisation term λ of 0.01. A unique solution $\hat{\boldsymbol{\theta}}^{r_2} = (\hat{\boldsymbol{H}} + 0.01 \cdot \boldsymbol{I}_b)^{-1} \hat{\boldsymbol{h}}$ could be solved in less than a second.

Sampling with Least Squares Once I had $\hat{\boldsymbol{\theta}}^{r_2}$, I was able to use to use PyTorchs autograd to track the operations from the kernel methods and dimension reduction functions. Then back-propagate them to calculate the conditional score. Let $\boldsymbol{X}_{\text{new}}^{\text{batch}} \in \mathbb{R}^{\text{batch} \times k}$, where batch is the batch size and k is the new dimension of the data. Due to the unconstrained formula allowing for negative $\hat{\boldsymbol{\theta}}^{r_2}$, after calculating the ratio, a minimum of 1e-8 is applied to all elements so the result is defined under a logarithm. Finally, the log-ratio is then summed to be able to use the autograd function.

$$\hat{\boldsymbol{s}}_t{}^{\text{nu}} = \nabla_{\boldsymbol{x}_t} \sum \max(X_{\text{new}}^{\text{batch}} \hat{\boldsymbol{\theta}}^{r_2}, \text{1e-8})$$

3.3 Diffusion and Guided Sampling

3.3.1 Diffusion Model Setup

Diffusion Model I used a linear variance schedule, which would start at $\beta_0 = 0.001$ and uniformly increase to $\beta_T = 0.02$. I chose the variance schedule due to its simplicity and the very small difference between the cosine schedule's performance. For all datasets, I used the same variance schedule setup for consistency among the results and simplicity. For the toy, structured and complex, I used 40, 500, and 1000 total timesteps, respectively. By increasing the number of timesteps, the model is able to have more steps to sample, and each step will have a lower variance of noise to be predicted, so overall helping the model sample from more difficult distributions.

Diffusion Neural Network For the toy data, I used a basic MLP with 5 linear layers, each with 64 weights. For the structured dataset, I used a U-Net with 3 down and up-sampling channels with one bottleneck layer. The feature maps were $\{64, 128, 256, 256, 256, 128, 64\}$, the timestep was added to the layer at each sampling channel. For the complex dataset (CIFAR-10), due to the computational cost and time to train a U-Net to give consistently good results to fairly compare the guidance methods. I used an open-to-use pre-trained U-Net from GitHub (w86763777, 2021).

3.3.2 Guidance Integration

Diffusion Model Guidance For the main formula for sampling from the conditional distribution, I used the practical version derived in section 2.2.2.

This form is better because it is simpler to implement and doesn't need the extra step of computing the score function directly.

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \hat{\boldsymbol{\epsilon}}(\boldsymbol{x}_t, \boldsymbol{\theta}^{\epsilon}) \right) + w\beta_t \nabla_{x_t} \log \hat{r}(\boldsymbol{x}_t, \boldsymbol{\theta}^{r}) + \beta_t \boldsymbol{z}$$

The guidance scale was chosen by maximising the F1 score, which is a harmonic mean of the precision and recall=.. This ensures the outputs maximise a combination of accuracy and diversity.

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Calculating Conditional Score To calculate the conditional score $\nabla_{x_t} \log \hat{r}(\boldsymbol{x}_t, \boldsymbol{\theta}^{r})$ from the density ratios, I used PyTorch's auto-grad function. In order to minimise the function, I summed the log-ratio over the batch size to get a single value. The auto-grad involved tracking all the operations that occurred to create the final summed log-ratio, so that the gradient could be back-propagated to the input, resulting in the conditional score.

Algorithm 3 Guided Diffusion Sampling via Density Ratio Estimation, given pre-trained parameters $\hat{\boldsymbol{\theta}}^{\epsilon}$ and $\hat{\boldsymbol{\theta}}^{r}$, let $\boldsymbol{\mu}_t$, $\tilde{\boldsymbol{\mu}}_t$ and $\boldsymbol{\epsilon}_t$ be function outputs for the mean, guided mean and error

Input: Diffusion model $\boldsymbol{\epsilon}(\boldsymbol{x}, t; \hat{\boldsymbol{\theta}}^{\epsilon})$, transformation function $\boldsymbol{\varphi}(\boldsymbol{x_t})$, uLSIF parameters $\hat{\boldsymbol{\theta}}^{r_2}$, logistic regressor $f(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^{r_1})$, maximum timestep T, guidance weight w, variance schedule variables $\beta_t, \alpha_t, \bar{\alpha}_t$
Sample from Gaussian noise: $\boldsymbol{x}_T \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$
for $t = T$ **to** 1 **do**
    Compute predicted noise: $\boldsymbol{\epsilon}_t \leftarrow \boldsymbol{\epsilon}(\boldsymbol{x}_t, t; \hat{\boldsymbol{\theta}}^{\epsilon})$
    Compute the standard mean: $\boldsymbol{\mu}_t \leftarrow \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon} \right)$
    if Least Squares Estimation (uLSIF) **then**
        Apply dimensionality reduction / basis function: $\boldsymbol{x}_{\text{new}} \leftarrow \boldsymbol{\varphi}(\boldsymbol{x}_t)$
        $\boldsymbol{s}_t^{\text{nu}} \leftarrow \nabla_{\boldsymbol{x}_t} \sum \log(\max(\boldsymbol{x}_{\text{new}} \hat{\boldsymbol{\theta}}^{r_2}, 10^{-8}))$
    else if Probabilistic Estimation (Logistic Regressor) **then**
        Compute logit: $z \leftarrow f(\boldsymbol{x}_t; \hat{\boldsymbol{\theta}}^{r_1})$
        $\boldsymbol{s}_t^{\text{nu}} \leftarrow \nabla_{\boldsymbol{x}_t} \sum z$
    end if
    Compute guided mean: $\tilde{\boldsymbol{\mu}}_t \leftarrow \boldsymbol{\mu}_t + w\beta_t \boldsymbol{s}_t^{\text{nu}}$
    if $t > 1$ **then**
        Sample from Gaussian noise $\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$
        $\boldsymbol{x}_{t-1} \leftarrow \tilde{\boldsymbol{\mu}}_t + \sqrt{\beta_t} \boldsymbol{z}$
    else
        $\boldsymbol{x}_0 \leftarrow \tilde{\boldsymbol{\mu}}_0$
    end if
end for
Output: $\boldsymbol{x}_t$ sampled from the conditional distribution

3.4 Evaluation Framework

3.4.1 Feature Extraction

My feature extractor for each dataset was a logistic regressor, which had two separated components: the features and output layers. The feature components included 3 convolutional layers, a flatten and a linear layer into 64 dimensions. The output layer was used to train the logistic regressor. When calculating score metrics, I would use the features part of the feature extractor.

3.4.2 Metrics

All the metrics, including FID, compared the real and fake features by inputting the real and generated images into the feature extractor. The feature extractors were custom-trained for the dataset, this is because I was using multiple different types of datasets. So a single feature extractor for all of the datasets would not have worked. The precision and recall used the k-nearest neighbours with $k = 6$, outlined from the paper(Kynkäänniemi et al., 2019). This dynamic radius can be used to account for high and low-density areas.

4 Experiments

This section includes an explanation of the datasets, quantitive metrics and qualitative examples so the effectiveness of each method explored can be fairly represented.

4.1 Dataset Analysis

4.1.1 Dataset Descriptions

Toy Data: Double Moons The double moons dataset consists of 2-dimensional data points that form a shape of two moons defined by parametric equations. Double Moons is part of a function in the Scikit-learn python library(Pedregosa et al., 2011). The dataset is very simple due to each variable only having two dimensions, so the dataset is good for testing the foundational model before moving to higher-dimensional datasets. There are two classes, the upper and lower moon; the data point can be generated, so there is no total size. For my investigations, I used 10,000 data points.

Structure Data: MNIST Numbers The MNIST Numbers dataset is a collection of $70,000$ hand-drawn images of numbers(LeCun et al., 1998). The dataset is labelled and includes the numbers from zero to nine. Each number is a 28 by 28 greyscale; the high-dimensionality and spatial components of the numbers make the dataset significantly harder than toy datasets. However, the consistency of the number always being centred and each number looking quite similar means MNIST Numbers can still function as simple dataset for

investigations. The dataset is split into 60,000 training images and 10,000 testing images.

Complex Data: CIFAR-10 CIFAR-10 contains 60,000 labelled images of real-world objects and animals(Krizhevsky, 2009). There are 6 different animals and 4 different vehicles; the images are 32 by 32 and RGB. The main object is not always in the centre or positioned in a predictable way. These qualities make CIFAR-10 complex and high-dimensional, closer to what would be found in real-world applications of guidance systems. The dataset has 50,000 training and 10,000 testing images.

4.1.2 Dataset Samples

Figure 1 shows an example image of each dataset. The Double Moons images contain 6,000 data points, the MNIST Numbers and CIFAR-10 examples both contain 25 randomly sampled images. The images present the large steps in complexity and dimensionality between each of the datasets.



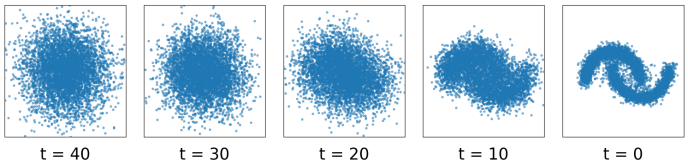
(a) Double Moons (b) MNIST Numbers (c) CIFAR-10

Figure 1: Samples from the data distribution of Double Moons, MNIST Numbers and CIFAR-10

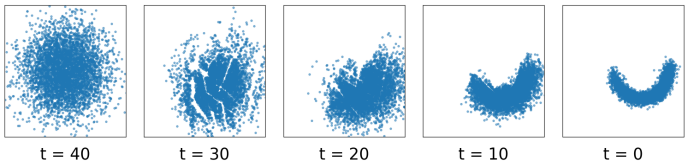
4.2 Double Moons

Qualitative

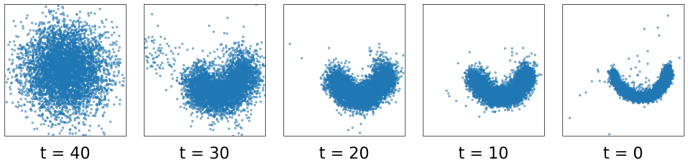
The figures show 3 different sampling trajectories, and the plot shows how the least squares and logistic regressor methods differ in how the data points are moved towards the lower moon. The logistic regressor seems to have less variance in the spread of points, whereas the least squares guidance method has less control.



(a) Unconditional Generation (Baseline)



(b) Logistic Regression Guidance (MLP)



(c) Least Squares Guidance (Gaussian Kernel)

Figure 2: Image comparison of the sampling process at different timesteps

Quantitative

The table 1 has the metrics xy-coverage, yx-coverage and MMD^2. I chose to use coverage because it is simple to use for point maps, being just a proportion of points that are close to the other distribution of points. By using both xy and yx, I was able to show how the accuracy and diversity change as the different conditioning methods are used. I chose MMD^2 to represent as an overall metric, which takes into account the accuracy and diversity.

Table 1: Comparing score metrics for 2D double moons guidance

Method	xy-cov ↑	yx-cov ↑	MMD^2 ↓
No conditioning	0.448 ± 5.8e-3	0.994 ± 6.0e-4	0.132 ± 2.9e-3
Logistic Regressor (MLP)	0.964 ± 1.1e-3	0.998 ± 1.0e-4	0.002 ± 1.0e-4
Least Squares (Kernel)	0.982 ± 1.0e-3	0.983 ± 1.1e-3	0.009 ± 3.0e-4

The xy-coverage increased by over 100% as nearly all the points were successfully steered towards the lower moon. The yx-coverage stayed very high, implying that even with the guidance, the diversity of the points did not decrease as a result of high accuracy. The MMD^2 at 0.002 of the Logistic Regressor was around 70% less than the Least squares method, which is also reflected in the figure 2 (c), where the image at $t = 0$ shows an image with not all the points

strictly in the region of the lower moon.

4.3 MNIST numbers

Qualitative

The four figures show outputs from three different guidance methods and a baseline. Overall, the methods successfully guide the diffusion model toward generating the digit 8 most of the time. The probabilistic logistic regressor achieves the most consistent outputs, whereas both least squares strategies occasionally produce incorrect digits. This inconsistency is expected due to the least squares formulation used in the uLSIF approach.



(a) Unconditional



(b) Logistic Regression



(c) Analytic (PCA)



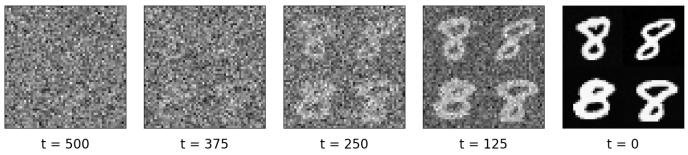
(d) Least Squares (Autoencoder)

Figure 3: Final MNIST generation results. Comparison between the unconditional baseline and conditional generation of the digit 8 using different guidance strategies.

Figure 4 shows an unconditional generation of the MNIST numbers dataset and three different methods for conditioning towards an eight. The three methods are a the probabilistic density ratio estimation using a CNN, uLSIF using PCA for dimensionality reduction and uLSIF using an autoencoder for dimensionality reduction.



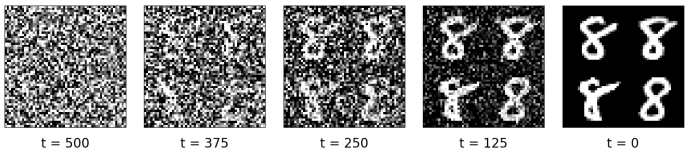
(a) Unconditional Generation (baseline)



(b) Logistic Regression Generation



(c) Least Squares Generation (PCA)



(d) Least Squares Generation (Autoencoder)

Figure 4: Image comparison between the unconditional and condition sampling process at different timesteps on MNIST numbers

Quantitative

The metrics I chose to assess the MNIST Numbers in table 2 were precision, recall and FID. Precision and recall are used to separate the accuracy and diversity, whilst the FID provides a general metric. The error values are computed as one standard deviation from the mean.

Table 2: Comparing guidance methods using Precision, Recall and FID

Method	Precision ↑	Recall ↑	FID ↓
No conditioning	0.2076 ± 1.5e-2	0.8728 ± 2.9e-2	86.909 ± 1.7
logistic regressor DRE guidance	0.7572 ± 2.3e-2	0.8564 ± 1.8e-2	17.882 ± 0.3
Least Squares (PCA)	0.7128 ± 2.5e-2	0.7980 ± 1.3e-2	20.582 ± 0.37
Least Squares (Autoencoder)	0.6864 ± 1.6e-2	0.8644 ± 1.6e-2	21.324 ± 0.6

All the guidance methods were able to decrease the FID by more than 75% and increase the precision by more than 400%, giving evidence of a successful application of the density ratio to guide the MNIST numbers to the number eight. The logistic regressor was clearly the best at guiding the diffusion model, achieving an FID of 17.9 and the best precision of 0.76, which is a sizeable improvement from the next best at 20.6 and 0.71 achieved by the least squares method using PCA as a dimensionality-reduction technique. The least squares method using an autoencoder as the dimensionality reducer performed the worst on both the precision and FID.

4.4 CIFAR-10

Qualitative

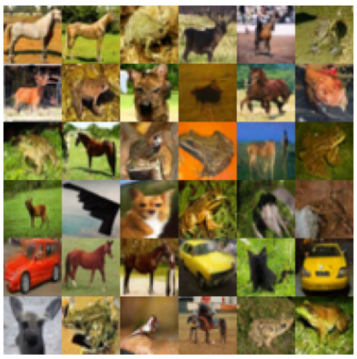
The next four images compare the unconditional baseline against three conditional generations of animals using different guidance methods.



(a) Unconditional (baseline)



(b) Logistic Regression



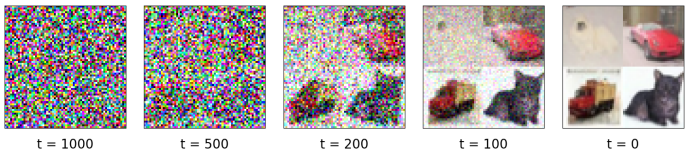
(c) Least Squares (PCA)



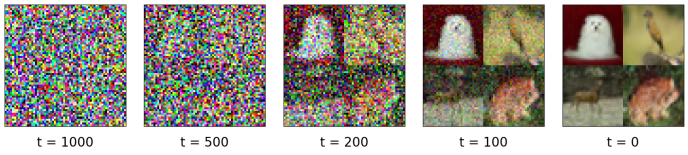
(d) Least Squares (Autoencoder)

Figure 5: Final generation results on CIFAR

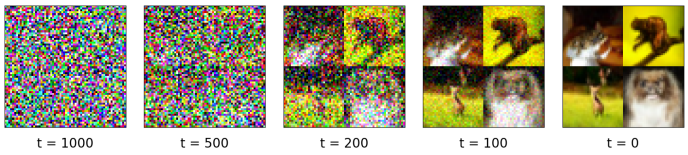
All 3 methods were able to guided most of generations to an animal, however the consistency dropped among all the methods due to the complexity of the CIFAR10 dataset.



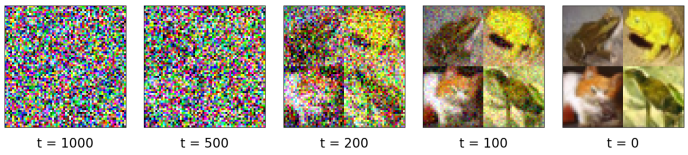
(a) Unconditional generation (baseline)



(b) Logistic Regressor Generation



(c) Least Squares Generation (PCA)



(d) Least Squares Generation (Latent)

Figure 6: Visualising the reverse diffusion timesteps on CIFAR10.

Quantitative

Table 3 use the metrics precision, recall and FID. The precision and recall are used to separate the accuracy and diversity components, and the FID represents an overall score.

Table 3: Comparison of guidance methods (Precision, Recall, FID, KID)

Method	Precision ↑	Recall ↑	FID ↓
No conditioning	0.79 ± 0.03	1.00 ± 0.009	5.20 ± 1.9
Logistic Regressor DRE guidance	0.96 ± 0.03	0.99 ± 0.01	2.13 ± 0.3
Least Squares (PCA)	0.95 ± 0.03	1.00 ± 0.009	1.96 ± 0.06
Least Squares (Autoencoder)	0.93 ± 0.05	0.99 ± 0.02	2.68 ± 0.6

Despite the complexity of CIFAR10, all the density ratio methods were able to halve the FID and increase the precision by more than 15%. The best FID at 1.96 was achieved by reducing the dimensionality by PCA. The Least Squares

method using the autoencoder had the worst result of an FID of 2.68 with the most variance.

4.5 Ablation Study

This ablation study will investigate the effectiveness of the guidance weight and visualise the trade-off between the image quality and diversity that occurs as the guidance weight increases.

Qualitative

I chose to use the precision and recall to present why single metrics like FID and MMD are not enough to present the whole picture of what is happening as the guidance weight increases.

Figure 7 shows the effect of increasing the guidance weight from 0 to 2.5. The images of '8's mid-way through the image have very diverse features, whereas by the last couple columns the '8's are much more similarly generated.

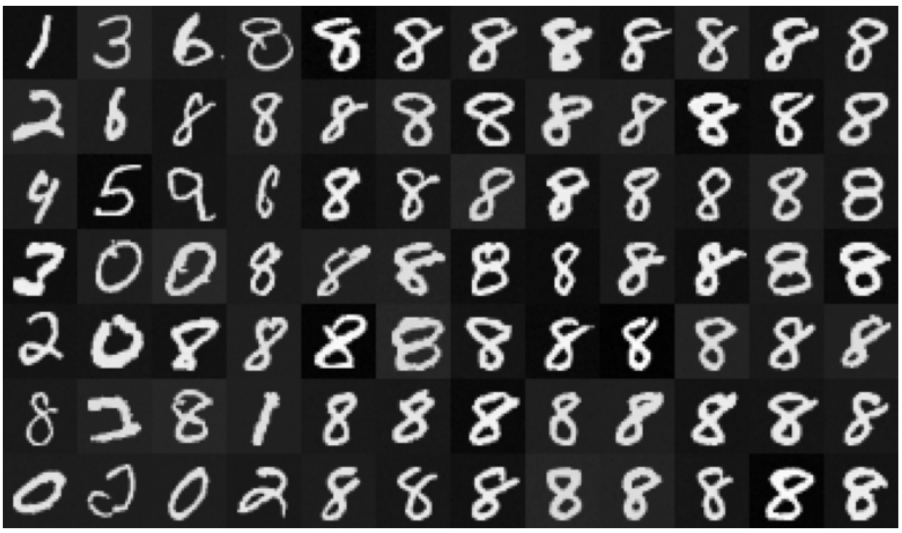


Figure 7: Diffusion output images as the guidance increases

Quantitative

Figure 8 includes two graphs, one has precision and recall, whilst the other graph has the F1 score. These metrics are chosen to separate the accuracy and diversity components of sampling. The F1 score is to show how a single metric can't give a full representation.

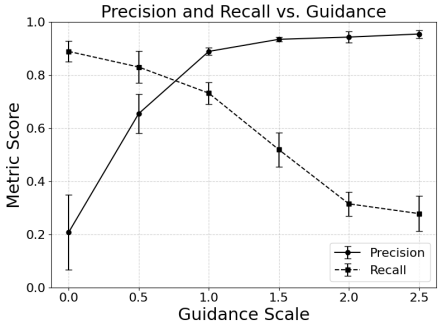


Figure 8: Precision and recall as the guidance weight increases

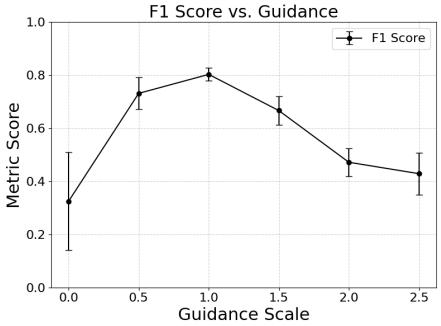


Figure 9: F1 score as the guidance weight increases

Figure 8 shows the precision and recall, and shows how a higher accuracy doesn't always mean the conditioning is better. As the guidance weights increase up to 2.5, the accuracy becomes very high only because the conditioned score is overpowering the unconditioned score, so only the most average sample of an 8 is being sampled. This is causing the recall to fall to 0.25, so to give the full picture, the recall must also be shown to represent the diversity.

Balancing the accuracy and diversity of the images is an important part of choosing the correct guidance value. The precision of the eight images has a large increase of 50% from increasing the guidance scale from 0 to 1, and then starts to converge. Past a guidance of 1, the small increase in precision is not worth the large decrease in diversity of the eight images. The F1 score reflects this notion by using a harmonic mean between the precision and recall. The F1 peaks at 0.8 when using a guidance scale of 1.

5 Discussion

5.1 Performance Comparison

On a purely performance-based standard, the probabilistic density estimator was the best at guiding the diffusion models, because the logistic regressor method was consistently the best or scored very close to the best on all levels of dataset difficulty. The consistency makes the logistic regressor a good one-for-all method. However, even though the least squares method was using an analytical solution, the method was still able to compete with the probabilistic density ratio estimator. This makes the least squares method a success because it represents a slight performance trade-off with a huge increase in efficiency.

The two methods for dimensionality reduction were principal component analysis (PCA) and an autoencoder (AE). The PCA method scored better on FID for both MNIST Numbers and CIFAR-10, (20.6 vs 21.3) and (1.96 vs 2.68), respectively. The PCA method is also much more efficient, because the components can be computed instantly, compared to the AE, which must be

trained to recreate the input images once.

5.2 Future Work

Throughout this paper, I showed how principal component analysis (PCA)(Jolliffe, 2002) can be used to reduce the dimensions before calculating the density ratio and hence steer the diffusion process to specific conditional distributions. However, in the paper(Yamada and Sugiyama, 2011) they propose a better method called Hetero-Distributional Subspace Analysis (HSA). In HSA, the Pearson Divergence between the numerator and denominator distributions is maximised compared to the variance, as in PCA. HSA is a more specialised analysis for density ratio estimation compared to PCA because the dimensionality can be reduced based on dissimilarity, compared to variance. I would like to research more about these specialised dimensionality reducers.

6 Conclusion

This report investigated the density ratio methods for conditioning diffusion models without training the base model. Through the theoretical derivation and different experiments, I was able to successfully steer the model towards the desired conditional distributions.

The experiments highlighted the importance of representation learning for the analytical density ratio. Whereas the probabilistic ratio estimation method gave a consistent method for all the levels of complexity. Among all the methods, least squares using principal component analysis to reduce the dimensions stood out not only as the most computationally efficient, due to its requiring zero pre-training, but it also could rival the performance of the probabilistic method.

References

Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis. *Advances in Neural Information Processing Systems*, 34:8780–8794, 2021.

Geoffrey E Hinton and Ruslan R Salakhutdinov. Reducing the dimensionality of data with neural networks. *science*, 313(5786):504–507, 2006.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

Ian T Jolliffe. Principal component analysis. *Encyclopedia of statistics in behavioral science*, 2002.

Takafumi Kanamori, Shohei Hido, and Masashi Sugiyama. A least-squares approach to direct importance estimation. *The Journal of Machine Learning Research*, 10:1391–1445, 2009.

Takafumi Kanamori, Taiji Suzuki, and Masashi Sugiyama. Theoretical analysis of density ratio estimation. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, E93-A(4):787–798, 2010. doi: 10.1587/transfun.E93.A.787.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. URL `https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf`.

Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved precision and recall metric for assessing generative models. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11): 2278–2324, 1998.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12: 2825–2830, 2011.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021. URL `https://openreview.net/forum?id=PxTIG12RRHS`.

Masashi Sugiyama, Taiji Suzuki, and Takafumi Kanamori. *Density ratio estimation in machine learning*. Cambridge University Press, 2012.

w86763777. pytorch-ddpm: Unofficial pytorch implementation of denoising diffusion probabilistic models. `https://github.com/w86763777/pytorch-ddpm`, 2021. Code and pretrained U-Net model weights released under the WTFPL license.

Makoto Yamada and Masashi Sugiyama. Direct density-ratio estimation with dimensionality reduction via hetero-distributional subspace analysis. In *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*, volume 25, pages 549–554, 2011.

A Appendix

A.1 uLSIF Least Squares Analytical Solution

Let $\boldsymbol{x}$ be samples from a data distribution $\boldsymbol{x} \in X$, and $\boldsymbol{\theta}^{r_2}$ be a vector of b parameters that will be optimised. Letting $p_{\text{de}}(\boldsymbol{x})$ and $p_{\text{nu}}(\boldsymbol{x})$ be the numerator and denominator probability density respectively. Then the density ratio is defined as

$$r^*(\boldsymbol{x}) = \frac{p_{\text{nu}}(\boldsymbol{x})}{p_{\text{de}}(\boldsymbol{x})}$$

Then the estimated density ratio $r(\boldsymbol{x}, \boldsymbol{\theta}^{r_2})$ be a linear combination of basis functions $\varphi_l(\boldsymbol{x})$,

$$r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2}) = \sum_{l=1}^{b} \theta_l^{r_2} \varphi_l(\boldsymbol{x})$$

To create a least squares solution, the loss objective must be a differentiable function, so the author uses the integral of the squared difference. This specific setup mirrors that of an expectation and will be helpful for the empirical approximation. Expanding the integral, and substituting in for $p_{\text{nu}}(\boldsymbol{x}) = r^*(\boldsymbol{x}) p_{\text{de}}$

$$\begin{aligned}
J_0(\boldsymbol{\theta}^{r_2}) &= \frac{1}{2} \int (r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2}) - r^*(\boldsymbol{x}))^2 p_{\text{de}}(\boldsymbol{x}) d\boldsymbol{x} \\
&= \frac{1}{2} \int \left[r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2})^2 p_{\text{de}}(\boldsymbol{x}) - 2 r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2}) r^*(\boldsymbol{x}) p_{\text{de}}(\boldsymbol{x}) + r^*(\boldsymbol{x})^2 p_{\text{de}}(\boldsymbol{x}) \right] d\boldsymbol{x} \\
&= \frac{1}{2} \int r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2})^2 p_{\text{de}}(\boldsymbol{x}) d\boldsymbol{x} - \int r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2}) p_{\text{nu}}(\boldsymbol{x}) d\boldsymbol{x} + \frac{1}{2} \int r^*(\boldsymbol{x})^2 p_{\text{de}}(\boldsymbol{x}) d\boldsymbol{x}
\end{aligned}$$

The last includes the theoretical true density ratio and does not include any terms dependent on theta $\boldsymbol{\theta}^{r_2}$, so it can be removed. Then the resulting formula is the main objective function in integral form.

$$J(\boldsymbol{\theta}^{r_2}) = \frac{1}{2} \int r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2})^2 p_{\text{de}}(\boldsymbol{x}) - \int r(\boldsymbol{x}; \boldsymbol{\theta}^{r_2}) p_{\text{nu}}(\boldsymbol{x}) d\boldsymbol{x}$$

The squared estimated density ratio function $r(\boldsymbol{x};\boldsymbol{\theta}^{r_2})^2$ can be expressed as a summation by substituting in for the summation $r(\boldsymbol{x};\boldsymbol{\theta}^{r_2}) = \sum_{l=1}^{b}\theta_l^{r_2}\varphi_l(\boldsymbol{x})$. The product of the two summations can then be written as a single summation.

$$\begin{aligned} r(\boldsymbol{x};\boldsymbol{\theta}^{r_2})^2 &= \left(\sum_{l=1}^{b}\theta_l^{r_2}\varphi_l(\boldsymbol{x})\right)\left(\sum_{l'=1}^{b}\theta_{l'}^{r_2}\varphi_{l'}(\boldsymbol{x})\right) \\ &= \sum_{l=1,l'=1}^{b}\theta_l^{r_2}\theta_{l'}^{r_2}\varphi_l(\boldsymbol{x})\varphi_{l'}(\boldsymbol{x}) \end{aligned}$$

Substituting the summation into $J(\boldsymbol{\theta}^{r_2})$, then the parameters θ_l^r don't depend on $\boldsymbol{x}$ and the integral can be factored inside the summation.

$$\begin{aligned} J(\boldsymbol{\theta}^{r_2}) &= \frac{1}{2}\int\left(\sum_{l=1,l'=1}^{b}\theta_l^{r_2}\theta_{l'}^{r_2}\varphi_l(\boldsymbol{x})\varphi_{l'}(\boldsymbol{x})\right)p_{\mathrm{de}}(\boldsymbol{x})d(\boldsymbol{x}) - \int\left(\sum_{l=1}^{b}\theta_l^{r_2}\varphi_l(\boldsymbol{x})\right)p_{\mathrm{nu}}(\boldsymbol{x})d\boldsymbol{x} \\ &= \frac{1}{2}\sum_{l=1,\ l'=1}^{b}\theta_l^{r_2}\theta_{l'}^{r_2}\left(\int\varphi_l(\boldsymbol{x})\varphi_{l'}(\boldsymbol{x})p_{\mathrm{de}}(\boldsymbol{x})\right) - \sum_{l=1}^{b}\theta_l^{r_2}\int\varphi_l(\boldsymbol{x})p_{\mathrm{nu}}(\boldsymbol{x})d\boldsymbol{x} \end{aligned}$$

Each integral takes the form of an expectation of basis functions $\varphi_l(\boldsymbol{x})$ and $\varphi_{l'}(\boldsymbol{x})$ over the continuous probability distributions. The expectations are not an analytical result. So, an approximation over the samples $\{x_i^{\mathrm{nu}}\}_{i=1}^{n^{\mathrm{nu}}}$ and $\{x_i^{\mathrm{de}}\}_{i=1}^{n^{\mathrm{de}}}$ is used. Where a vector $\boldsymbol{h}$ and matrix $\boldsymbol{H}$ are indexed by the index of basis functions $\boldsymbol{\varphi}(\boldsymbol{x})$, and the approximated expectations are $\hat{\boldsymbol{H}}$ and $\hat{\boldsymbol{h}}$

$$\begin{aligned} \boldsymbol{h}_l = \int\varphi_l(\boldsymbol{x})p_{\mathrm{nu}}(\boldsymbol{x})d\boldsymbol{x} &= \mathbb{E}_{\boldsymbol{x}\sim p_{\mathrm{nu}}(\boldsymbol{x})}\left[\varphi_l(\boldsymbol{x})\right] \\ \hat{h}_{l,l'} &= \frac{1}{n^{\mathrm{nu}}}\sum_{i=1}^{n^{\mathrm{nu}}}\varphi_l(\boldsymbol{x}_i^{nu}) \\ \boldsymbol{H_l\phi} = \int\varphi_l(\boldsymbol{x})\varphi_{l'}(\boldsymbol{x})p_{\mathrm{de}}(\boldsymbol{x})d\boldsymbol{x} &= \mathbb{E}_{\boldsymbol{x}\sim p_{\mathrm{de}}(\boldsymbol{x})}\left[\varphi_l(\boldsymbol{x})\varphi_{l'}(\boldsymbol{x})\right] \\ \hat{H}_{l,l'} &= \frac{1}{n^{\mathrm{de}}}\sum_{i=1}^{n^{\mathrm{de}}}\varphi_l(\boldsymbol{x}_i^{\mathrm{de}})\varphi_{l'}(\boldsymbol{x}_i^{\mathrm{de}}) \end{aligned}$$

Each summation can be represented as a matrix multiplication between the $\boldsymbol{\theta}^{r_2}$, $\hat{\boldsymbol{H}}$ and $\hat{\boldsymbol{h}}$, giving the matrix form of the loss objective $J(\boldsymbol{\theta}^{r_2}) = \frac{1}{2}(\boldsymbol{\theta}^{r_2})^\top\hat{\boldsymbol{H}}\boldsymbol{\theta}^{r_2} - \hat{\boldsymbol{h}}^\top\boldsymbol{\theta}^{r_2}$

A regularisation term λ is introduced, which, when multiplied by $\boldsymbol{I}_b^T\boldsymbol{\theta}^{r_2}$, results in the λ multiplied by the sum of terms. The density ratio must be non-negative, so the least important squared fitting has the loss objective.

$$\min_{(\boldsymbol{\theta}^{r_2})\in\mathbb{R}^b}\left[\frac{1}{2}(\boldsymbol{\theta}^{r_2})^\top\hat{\boldsymbol{H}}\boldsymbol{\theta}^{r_2} - \hat{\boldsymbol{h}}^\top + \lambda I_b^\top\boldsymbol{\theta}^{r_2}\right],\ \boldsymbol{\theta}^{r_2} \geq \mathbf{0}_b$$

However, the constraint is not yet analytical with the non-negativity constraint, so the constraint is ignored. To avoid breaking the regularisation term

λ, the square of each parameter in $\boldsymbol{\theta}^{r_2}$ is used $(\boldsymbol{\theta}^{r_2})^\top \boldsymbol{\theta}^{r_2}$, so the convex minimum still exists.

$$\min_{\boldsymbol{\theta}^{r_2} \in \mathbb{R}^b} \left[\frac{1}{2} (\boldsymbol{\theta}^{r_2})^\top \hat{\boldsymbol{H}} \boldsymbol{\theta}^{r_2} - \hat{\boldsymbol{h}}^\top \boldsymbol{\theta}^{r_2} + \frac{1}{2} \lambda (\boldsymbol{\theta}^{r_2})^\top \boldsymbol{\theta}^{r_2} \right]$$

Calculating the gradient with respect to the parameters $\boldsymbol{\theta}$ derives the final analytical solution.

$$\nabla J(\theta) = \nabla \left(\frac{1}{2} \boldsymbol{\theta}^\top \hat{\boldsymbol{H}} \boldsymbol{\theta} - \hat{\boldsymbol{h}}^\top \boldsymbol{\theta} + \frac{\lambda}{2} \boldsymbol{\theta}^\top \boldsymbol{\theta} \right) = \boldsymbol{H} \boldsymbol{\theta} - \hat{\boldsymbol{h}} + \lambda \boldsymbol{\theta}$$

Solving for 0 then gives results in the analytical solution $\hat{\boldsymbol{\theta}}$

$$\boldsymbol{H} \hat{\boldsymbol{\theta}}^{r_2} - \hat{\boldsymbol{h}} + \lambda \hat{\boldsymbol{\theta}}^{r_2} = 0 \Rightarrow \hat{\boldsymbol{\theta}}^{r_2} (\boldsymbol{H} + \lambda I) = \hat{\boldsymbol{h}} \Rightarrow \hat{\boldsymbol{\theta}}^{r_2} = (\hat{\boldsymbol{H}} + \lambda I)^{-1} \hat{\boldsymbol{h}}$$